Lab 1: Greedy heuristics

Authors: Aleksander Hański, Kacper Kuźnik

Source code: https://github.com/aleksanderhanski/evolutionary-computation/tree/main/lab1-greedy-heuristics

1. Description of the problem

We are given a set of nodes. Each node has x and y coordinates in a plane and a cost. The goal is to select exactly 50% of the nodes (rounded up if the number of nodes is odd) and form a Hamiltonian cycle through the selected nodes, so that the sum of the total length of the cycle and the total cost of the selected nodes is minimized.

Distances between nodes are Euclidean distances rounded mathematically to integers. The distance matrix is computed once, right after reading an instance. The optimization methods only use the distance matrix and the vector of node costs, never the coordinates.

For a solution given as an ordered list of nodes `path = [v_0, ..., v_(k-1)]`:

```
None
objective(path) = sum over i of D[v_i][v_((i+1) mod k)]  +  sum over i of
cost[v_i]
```

Both instances, TSPA and TSPB, have 200 nodes, so every solution contains k = 100 nodes.

2. Pseudocode of the implemented algorithms

Notation: `D` is the distance matrix, `cost` the node costs, `n` the number of nodes, `k = ceil(n / 2)`. "Nearest" is never the plain distance: every method picks the move with the best change of the objective function, i.e. added edge length plus the cost of the added node. Ties are broken by the first position and then the lowest node index.

Random solution

```
None
random_solution(D, cost):
    path := k distinct nodes drawn uniformly at random, in random order
    return path
```

Nearest neighbor, adding only at the end of the path

```
None
nn_end_only(D, cost, start):
    path := [start]
    while |path| < k:
        last := last node of path
        for each node v not in path:
            delta(v) := D[last][v] + cost[v]
        append the node v with the smallest delta(v) to the end of path
    return path
```

Nearest neighbor, adding at any position

The path is open while it is being built. The edge closing the cycle only appears in the final solution, so it is not part of the evaluation.

```
None
nn_any_position(D, cost, start):
    path := [start]
    while |path| < k:
        for each node v not in path:
            for each position p in 0 .. |path|:
                if p = 0:                                   # before the first
node
                    delta(v, p) := D[v][path[0]] + cost[v]
                else if p = |path|:                         # after the last node
                    delta(v, p) := D[path[last]][v] + cost[v]
                else:                                       # between a =
path[p-1] and b = path[p]
                    delta(v, p) := D[a][v] + D[v][b] - D[a][b] + cost[v]
        insert the node v at the position p with the smallest delta(v, p)
    return path
```

Greedy cycle

The same insertion rule, but the solution is a closed cycle all the time, so the edge between the last and the first node is also a candidate for insertion.

```
None
greedy_cycle(D, cost, start):
    cycle := [start]
    while |cycle| < k:
        for each node v not in cycle:
            for each edge (a, b) of the cycle, including (last, first):
                delta(v, a, b) := D[a][v] + D[v][b] - D[a][b] + cost[v]
        insert the node v between a and b with the smallest delta(v, a, b)
    return cycle
```

With a single node the cycle has one degenerate edge `(start, start)` of length 0, so the first step chooses the node minimizing `2 * D[start][v] + cost[v]`.

3. Results of the computational experiment

For each greedy method 200 solutions were generated, one starting from each node. 200 random solutions were generated with a fixed seed (42). Values are min / average / max of the objective function.

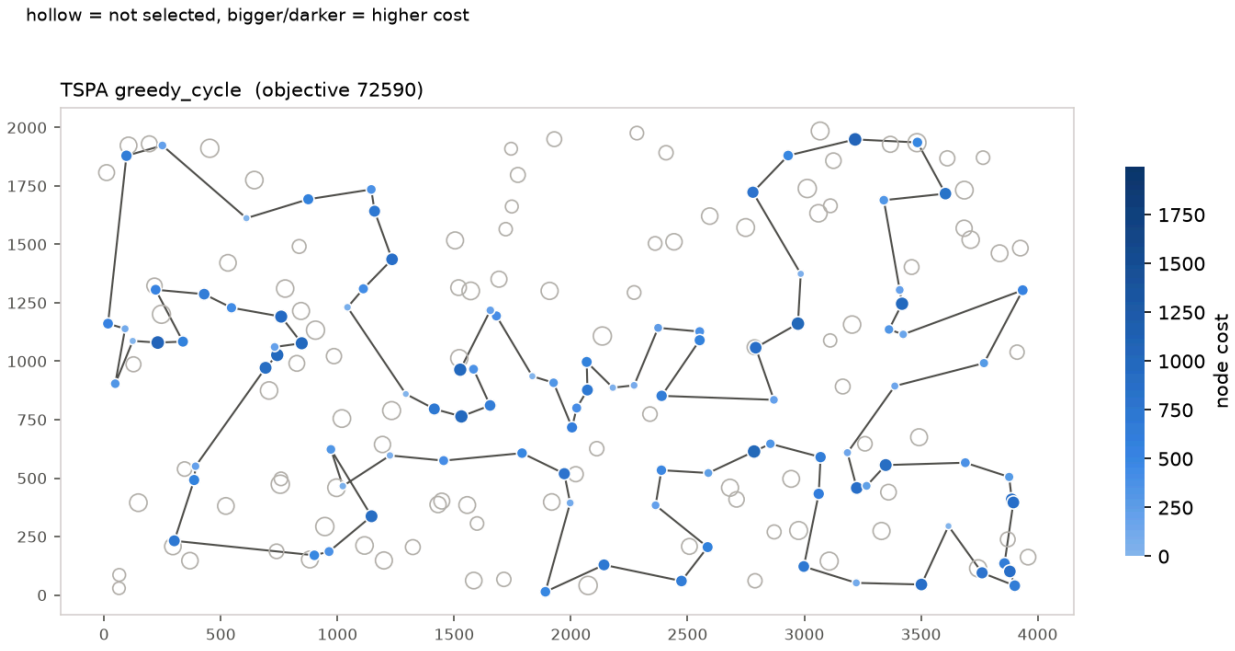
Method	**TSPA min**	**TSPA avg**	**TSPA max**
Random	240892	262956.1	289850
Nearest neighbor (end only)	84362	87476.6	90075
Nearest neighbor (any position)	**72328**	74306.5	77022
Greedy cycle	72590	**73743.7**	**75281**

Method	**TSPB min**	**TSPB avg**	**TSPB max**
Random	186840	206333.8	227175
Nearest neighbor (end only)	56007	**59397.3**	**63166**
Nearest neighbor (any position)	**52915**	59476.2	66164
Greedy cycle	55655	59650.8	63734

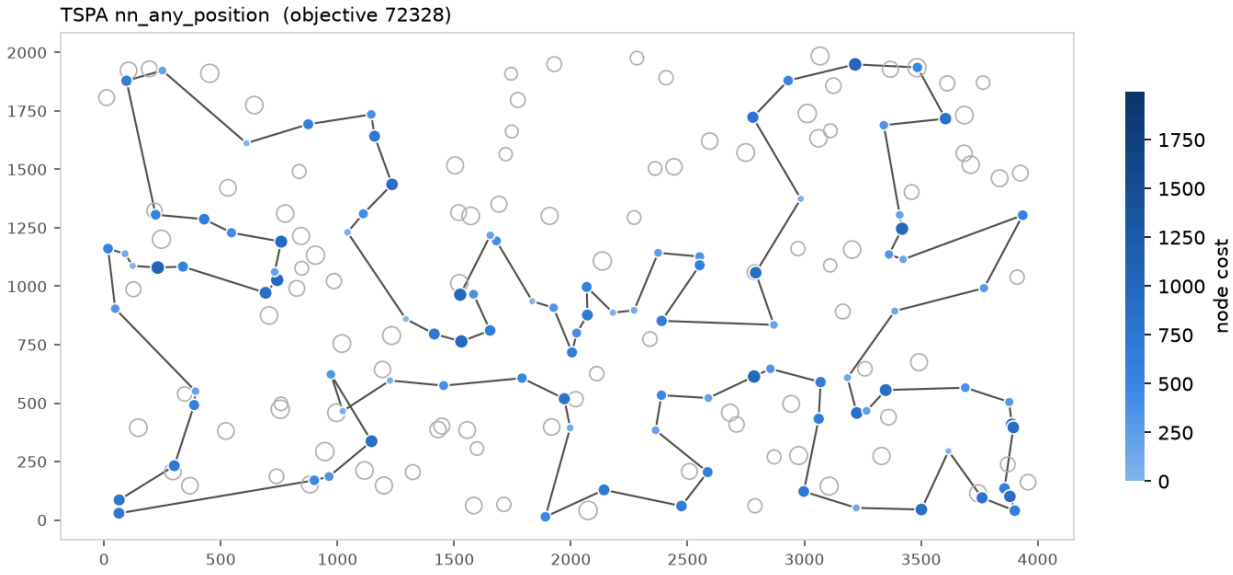
4. Visualization of the best solutions

Each panel shows the best solution of one method. Selected nodes are filled, and both their size and color show the node cost (bigger and darker means more expensive). Nodes that are not selected are drawn as hollow grey circles, with size still showing their cost.

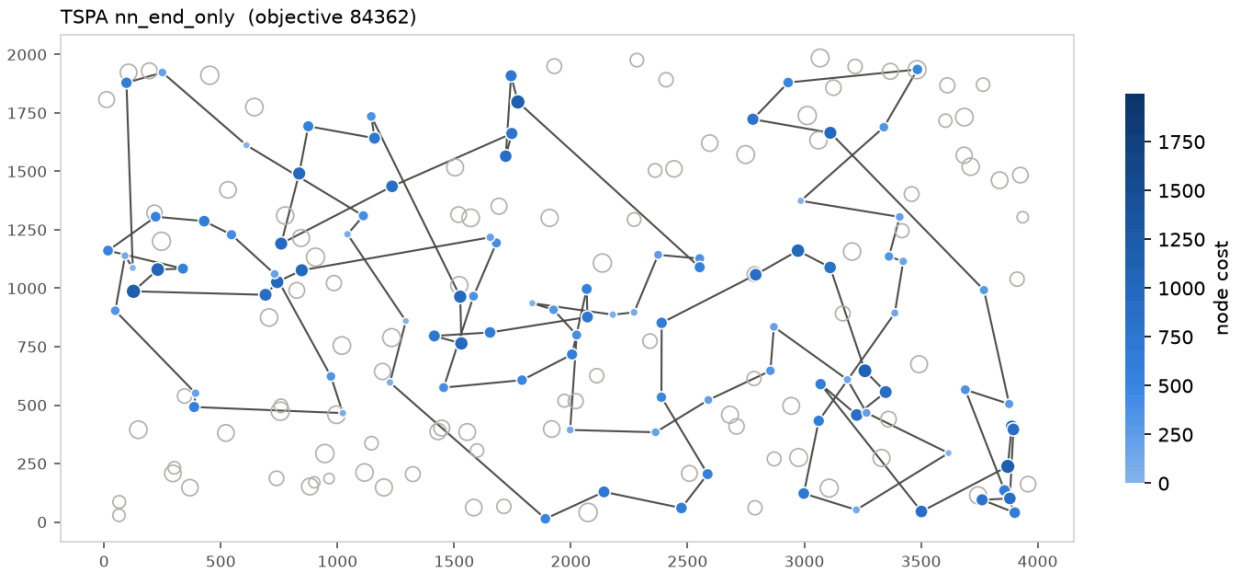
TSPA



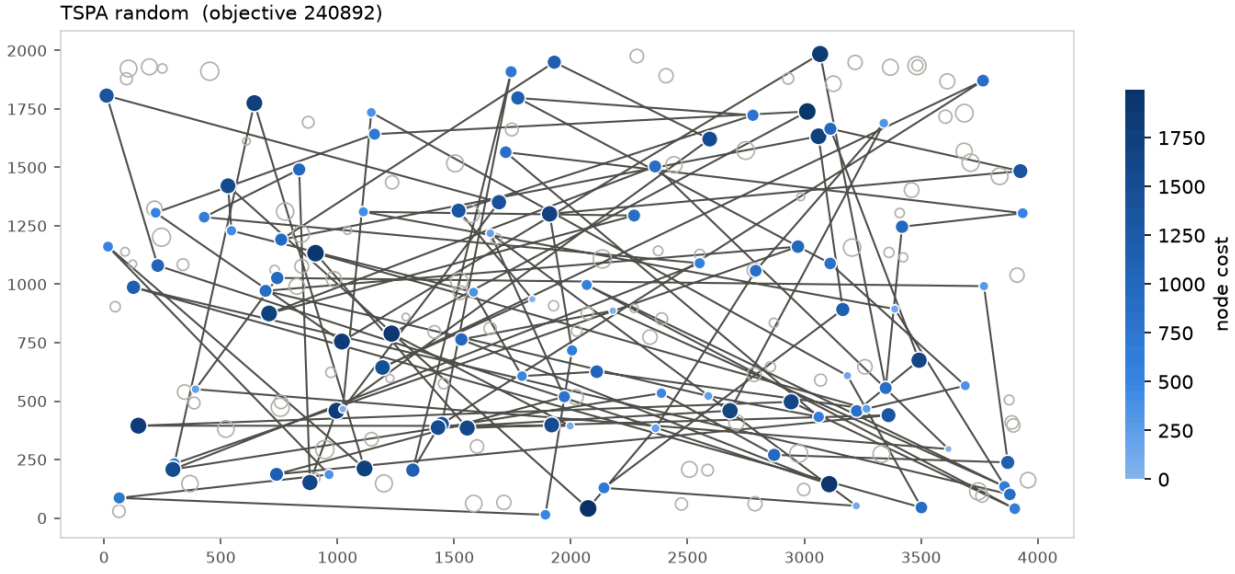
hollow = not selected, bigger/darker = higher cost



hollow = not selected, bigger/darker = higher cost

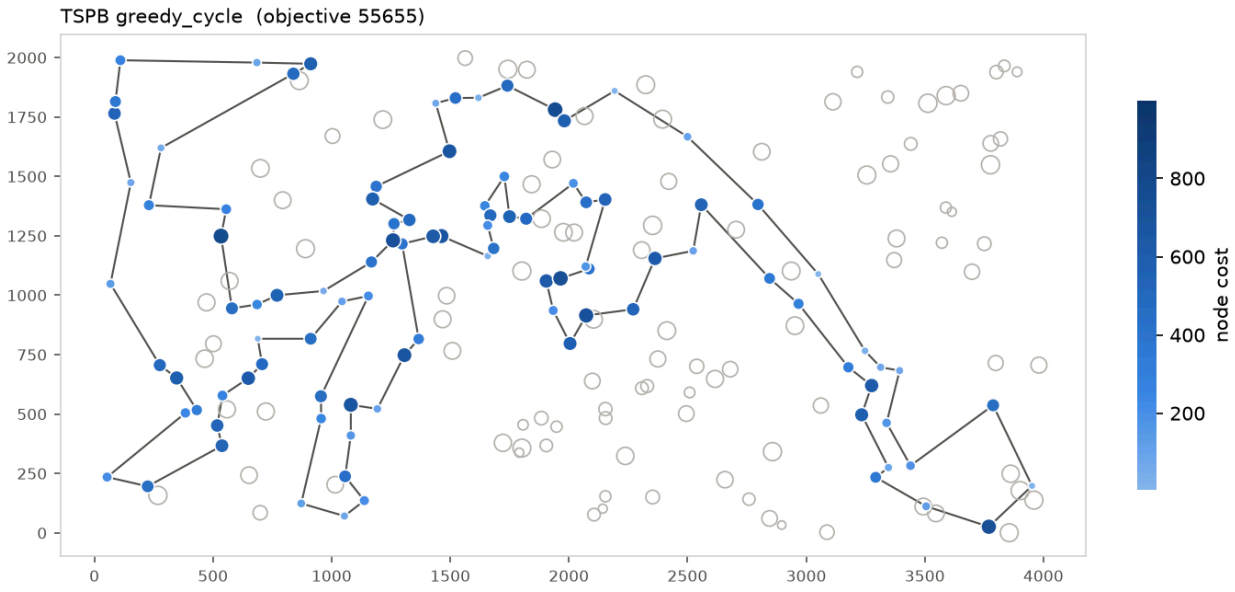


hollow = not selected, bigger/darker = higher cost

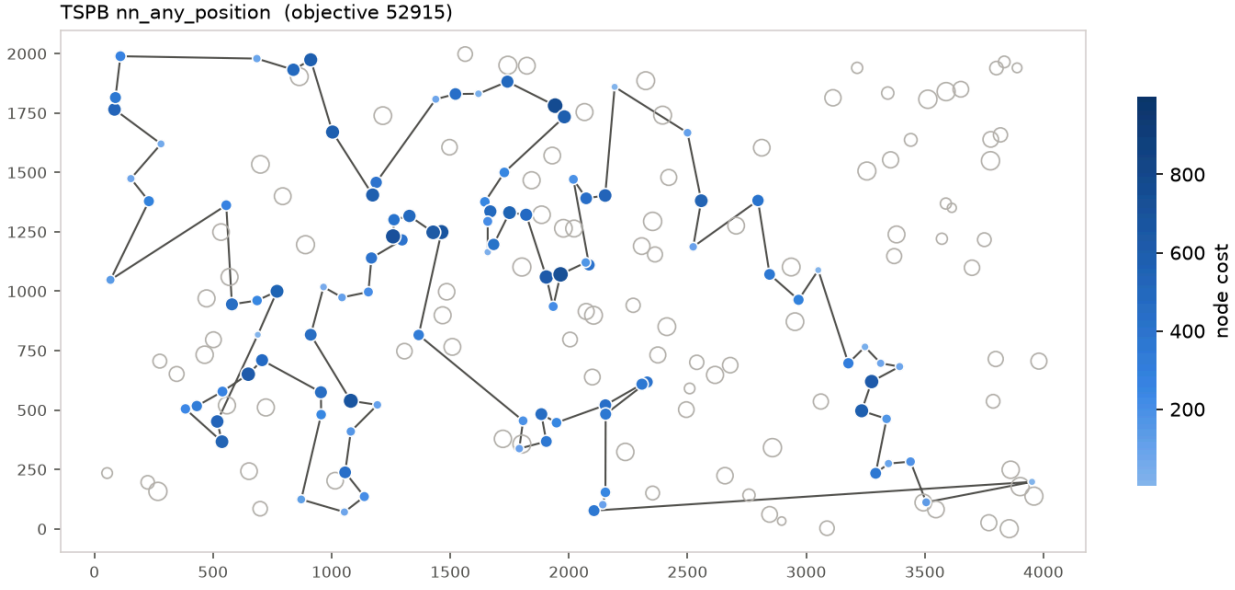


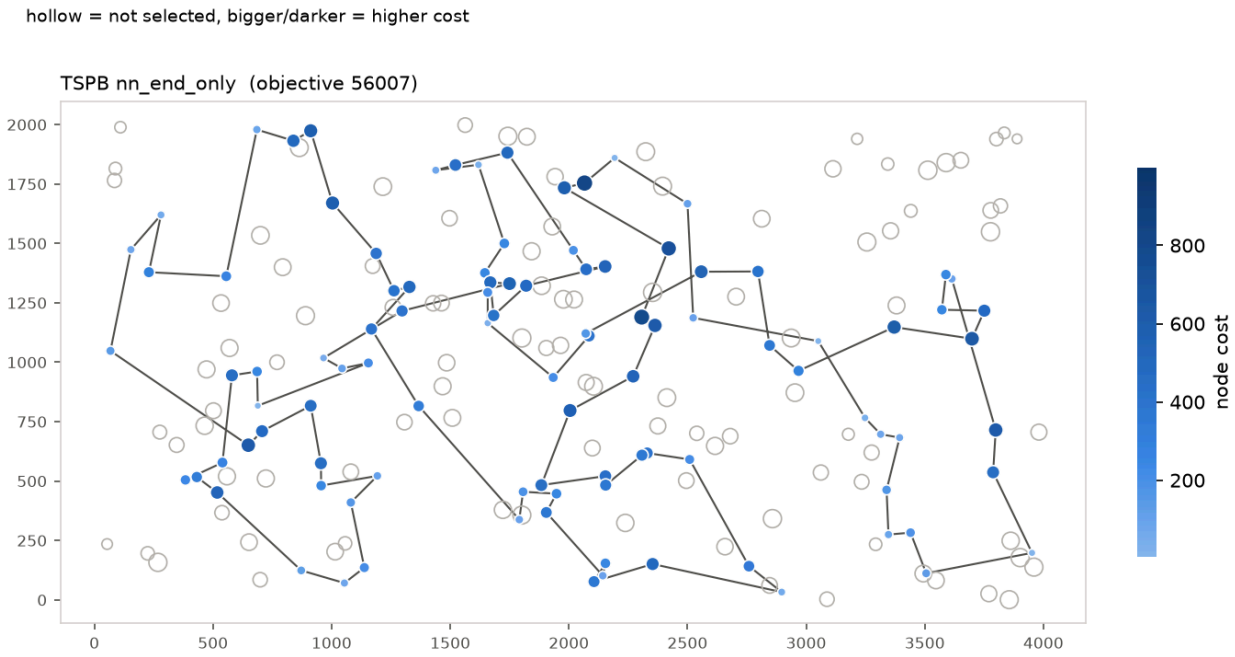
TSPB

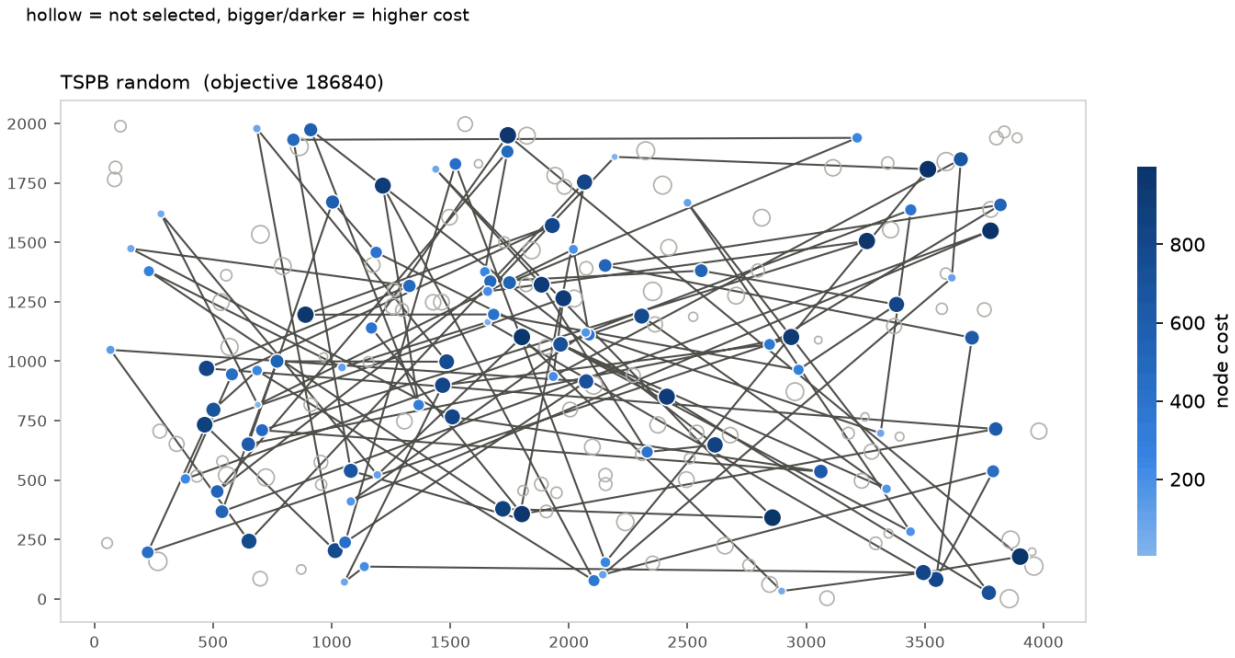
hollow = not selected, bigger/darker = higher cost



hollow = not selected, bigger/darker = higher cost







5. Best solutions

Lists of node indices, starting from 0, in the order they are visited. The cycle closes from the last node back to the first.

TSPA

Random (objective 240892)

```
None
12, 143, 76, 93, 193, 119, 64, 106, 157, 187, 133, 77, 11, 105, 102, 177, 153, 52,
78, 0, 188, 135, 191, 195, 71, 60, 23, 197, 113, 169, 147, 139, 151, 134, 160,
142, 69, 137, 166, 116, 2, 129, 114, 140, 190, 21, 184, 117, 42, 130, 85, 5, 186,
196, 67, 145, 55, 27, 115, 10, 96, 13, 178, 8, 58, 118, 66, 152, 35, 45, 41, 89,
68, 70, 121, 103, 92, 73, 51, 57, 181, 81, 56, 100, 175, 34, 40, 32, 38, 108, 54,
94, 84, 138, 75, 47, 83, 97, 107, 62
```

Nearest neighbor (end only) (objective 84362)

```
None
54, 9, 33, 36, 177, 67, 144, 53, 135, 58, 134, 176, 174, 106, 89, 107, 57, 182,
151, 93, 50, 3, 1, 164, 178, 29, 136, 186, 184, 47, 159, 130, 153, 78, 88, 193,
170, 146, 75, 64, 150, 42, 113, 43, 85, 142, 8, 131, 84, 158, 192, 181, 114, 141,
180, 132, 76, 80, 48, 31, 63, 149, 168, 97, 127, 187, 30, 123, 152, 108, 60, 116,
49, 11, 119, 160, 83, 90, 51, 16, 35, 87, 45, 140, 52, 179, 185, 28, 102, 156,
157, 71, 91, 38, 100, 4, 112, 139, 115, 5
```

Nearest neighbor (any position) (objective 72328)

```
None
104, 129, 69, 127, 97, 168, 116, 149, 63, 11, 49, 160, 83, 123, 100, 152, 108, 60,
31, 48, 80, 91, 157, 71, 4, 76, 132, 180, 28, 102, 185, 35, 156, 51, 16, 67, 144,
140, 53, 179, 52, 177, 36, 33, 9, 54, 131, 174, 8, 164, 186, 136, 163, 29, 101,
178, 1, 169, 3, 50, 147, 47, 93, 106, 42, 89, 113, 130, 159, 146, 170, 153, 193,
78, 88, 107, 64, 57, 182, 151, 150, 176, 72, 134, 84, 58, 158, 192, 181, 114, 135,
73, 45, 87, 141, 187, 30, 189, 2, 171
```

Greedy cycle (objective 72590)

```
None
31, 116, 149, 168, 63, 11, 49, 60, 108, 152, 100, 90, 123, 83, 160, 97, 127, 69,
171, 2, 189, 30, 187, 141, 87, 45, 73, 135, 114, 181, 192, 158, 58, 84, 134, 72,
176, 150, 151, 182, 57, 64, 107, 88, 78, 193, 153, 170, 146, 159, 130, 113, 89,
42, 106, 93, 47, 147, 50, 3, 169, 1, 178, 101, 29, 163, 136, 186, 164, 142, 8,
```

```
174, 131, 54, 9, 33, 36, 177, 52, 179, 53, 140, 144, 67, 16, 51, 156, 35, 185,
102, 28, 180, 132, 76, 4, 71, 157, 91, 80, 48
```

TSPB

Random (objective 186840)

None
```
69, 140, 145, 107, 186, 121, 95, 24, 119, 42, 64, 195, 82, 125, 19, 174, 159, 180,
50, 130, 35, 93, 57, 14, 173, 153, 75, 132, 177, 166, 161, 160, 192, 41, 185, 136,
197, 48, 133, 8, 190, 147, 141, 85, 32, 91, 65, 111, 157, 198, 179, 199, 89, 172,
12, 115, 150, 49, 28, 183, 53, 56, 101, 38, 16, 100, 2, 171, 128, 40, 149, 170,
72, 3, 129, 187, 184, 60, 193, 36, 26, 124, 116, 142, 102, 109, 151, 169, 114, 87,
118, 9, 143, 86, 61, 162, 4, 134, 45, 188
```

Nearest neighbor (end only) (objective 56007)

None
```
32, 89, 57, 76, 54, 127, 129, 90, 153, 20, 14, 2, 110, 118, 56, 114, 196, 10, 148,
143, 78, 192, 162, 104, 185, 184, 3, 151, 69, 172, 133, 120, 96, 121, 198, 48, 72,
46, 86, 117, 160, 136, 8, 21, 154, 140, 47, 170, 125, 177, 6, 132, 29, 197, 175,
134, 45, 60, 40, 71, 155, 97, 115, 180, 65, 28, 149, 145, 158, 116, 147, 42, 50,
130, 30, 183, 38, 64, 85, 59, 167, 51, 87, 179, 82, 88, 171, 99, 67, 24, 34, 181,
152, 73, 98, 27, 135, 75, 79, 156
```

Nearest neighbor (any position) (objective 52915)

None
```
110, 2, 14, 20, 74, 153, 182, 52, 90, 129, 127, 80, 54, 192, 162, 104, 76, 185,
57, 89, 86, 46, 72, 3, 184, 173, 151, 122, 117, 8, 160, 69, 172, 136, 133, 120,
156, 43, 48, 96, 198, 121, 130, 191, 50, 42, 147, 116, 37, 146, 13, 149, 28, 145,
65, 158, 177, 125, 26, 170, 197, 36, 132, 29, 6, 180, 115, 155, 71, 175, 134, 45,
157, 60, 40, 141, 97, 154, 140, 47, 38, 21, 176, 30, 183, 189, 168, 64, 59, 85,
51, 73, 167, 152, 34, 24, 181, 179, 87, 82
```

Greedy cycle (objective 55655)

```
None
174, 36, 197, 6, 180, 115, 170, 97, 140, 47, 155, 71, 175, 134, 45, 157, 60, 141,
40, 33, 64, 21, 189, 168, 69, 160, 172, 136, 133, 120, 8, 117, 72, 46, 86, 184, 3,
173, 122, 151, 98, 111, 27, 135, 76, 185, 162, 192, 80, 52, 182, 20, 74, 2, 93,
110, 118, 14, 153, 90, 129, 127, 54, 104, 57, 89, 156, 43, 48, 96, 198, 121, 63,
130, 191, 183, 30, 176, 38, 154, 26, 125, 177, 138, 158, 145, 149, 147, 42, 116,
37, 146, 13, 28, 65, 178, 103, 29, 132, 1
```

6. Solution checker

All eight best solutions were checked

Instance	Method	Our objective	Checker objective	Match
TSPA	Random	240892	240892	yes
TSPA	Nearest neighbor (end only)	84362	84362	yes
TSPA	Nearest neighbor (any position)	72328	72328	yes
TSPA	Greedy cycle	72590	72590	yes
TSPB	Random	186840	186840	yes
TSPB	Nearest neighbor (end only)	56007	56007	yes
TSPB	Nearest neighbor (any position)	52915	52915	yes
TSPB	Greedy cycle	55655	55655	yes

7. Conclusions

- All three greedy heuristics are far better than random solutions: the average objective is about 3 to 3.5 times lower on both instances.
- Allowing insertion at any position clearly helps on TSPA. Compared with adding only at the end of the path, the average drops from 87476.6 to 74306.5 for nearest neighbor (any position) and to 73743.7 for greedy cycle, about 15%. Adding only at the end leaves long edges and crossings, which is visible in the plot.
- On TSPA nearest neighbor (any position) found the single best solution (72328), while greedy cycle has the best average and the best worst case, so it is the more reliable of the two.

- On TSPB the three greedy methods are almost equal on average (59397.3, 59476.2 and 59650.8, within 0.5%). Nearest neighbor (any position) still found the best solution (52915), but it also has the widest spread (52915 to 66164), so its result depends strongly on the start node.
- The node cost matters as much as the distance: all greedy methods mostly skip expensive nodes, which is why their solutions are built from the small, light nodes in the plots.
- The better methods are slower. Adding only at the end takes about 0.4 ms per solution, while the two insertion methods take about 32 ms per solution in our Python implementation, because every step evaluates every free node at every position.